Q1. Define Process and Explain Process States with a Neat Diagram

---

1. Definition of Process

A **process** is a program in execution. It is an **active entity** that includes both the program code and its current execution state.

A process consists of:

- Program code
- Program Counter (address of next instruction)
- CPU registers
- Process stack (local variables, function calls)
- Allocated memory (heap, data section)
- Associated resources (files, I/O devices)

A program is a **passive entity**, whereas a process is an **active entity executing in memory**

Example

A stored application file is a program. When it is executed (e.g., opening a browser), it becomes a process.

---

2. Process States

As a process executes, it changes its state based on its activity and resource availability. The commonly used model is the **five-state process model**.

1. New State

- The process is being created
- The operating system initializes its Process Control Block (PCB)

2. Ready State

- The process is loaded into main memory
- It is ready to execute but waiting for CPU allocation

3. Running State

- The process is currently being executed by the CPU

4. Waiting (Blocked) State

- The process is waiting for an event such as I/O completion
- It cannot proceed until the event occurs

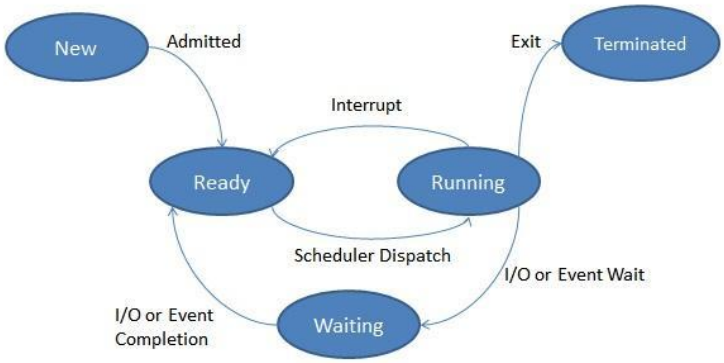
5. Terminated State

- The process has completed execution
- The operating system deallocates its resources

These states represent the complete lifecycle of a process

---

3. Process State Diagram



4. State Transitions

The transitions between states are managed by the operating system scheduler:

- **New → Ready (Admitted):**
  The process is created and placed in main memory
- **Ready → Running (Scheduler Dispatch):**
  The CPU scheduler selects the process for execution
- **Running → Waiting (I/O or Event Wait):**
  The process requests I/O or waits for an event
- **Waiting → Ready (Event Completion):**
  After the event completes, the process returns to the ready queue
- **Running → Ready (Preemption):**
  The process is interrupted due to time slice completion or higher priority process
- **Running → Terminated (Exit):**
  The process completes execution

---

5. Tabular Summary of Process States

State	Description	Location
New	Process is being created	Main Memory
Ready	Waiting for CPU	Main Memory
Running	Executing instructions	CPU
Waiting	Waiting for I/O or event	Memory / I/O Queue
Terminated	Execution completed	Removed from system

---

6. Example

Consider a program execution:

1. A user starts an application → **New**
2. The application is loaded into memory → **Ready**
3. CPU starts executing → **Running**
4. The application requests file input → **Waiting**
5. File operation completes → **Ready** → **Running**
6. Execution finishes → **Terminated**

---

7. Conclusion

Process states describe the lifecycle of a process from creation to termination.
They enable the operating system to manage CPU scheduling, resource allocation, and multitasking efficiently.

Q2. Explain Process Control Block (PCB)

---

1. Definition

A **Process Control Block (PCB)** is a data structure used by the operating system to store and manage all information related to a process.

Each process is assigned a unique PCB when it is created, and it is used throughout the process lifecycle to control its execution

2. Contents of PCB

The PCB stores the following information:

1. Process Identification

- Process ID (PID)

2. Process State

- Current state (Ready, Running, Waiting, etc.)

3. Program Counter

- Address of next instruction to execute

4. CPU Registers

- Stores register values during execution

5. CPU Scheduling Information

- Process priority
- CPU time (quantum)
- Scheduling queue details

6. Memory Management Information

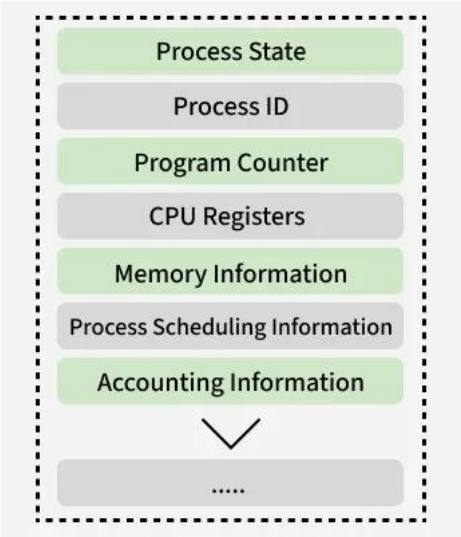
- Page tables or memory limits

7. I/O and File Information

- List of open files
- Allocated devices

---

3. PCB Diagram



4. Working of PCB

- When a process is created, the OS creates a PCB
- During execution, the PCB is updated continuously
- During context switching:
  - Current process data is saved in PCB
  - Next process data is loaded from PCB
- When the process resumes, it continues from the saved state

---

5. Applications of PCB

- Helps in CPU scheduling
- Enables context switching
- Tracks process state and execution
- Supports resource management and allocation

---

6. Conclusion

The PCB is an essential component of process management, as it maintains all necessary information required for execution, scheduling, and control of a process.

Q3. Explain Process Scheduling: Types of Schedulers and Scheduling Criteria

1. Definition of Process Scheduling

Process scheduling is the activity of the operating system that decides which process should be executed by the CPU at a given time.

It ensures efficient utilization of CPU and improves overall system performance by managing multiple processes.

---

2. Types of Process Schedulers

1. Long-Term Scheduler (Job Scheduler)

- Selects processes from the **job pool (secondary storage)**
- Loads them into main memory (ready queue)

- Controls the **degree of multiprogramming**
- Invoked **infrequently**

Example: Selecting programs from disk to load into memory

2. Short-Term Scheduler (CPU Scheduler)

- Selects a process from the **ready queue**
- Allocates CPU for execution
- Invoked **very frequently**
- Must be **fast**

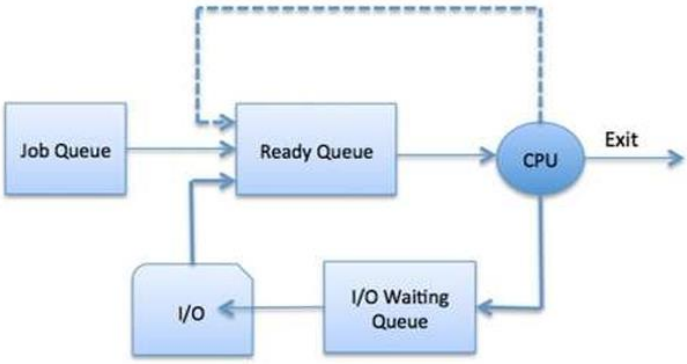
Example: Choosing the next process to run on CPU

3. Medium-Term Scheduler

- Suspends and resumes processes
- Performs **swapping** (memory management)
- Helps reduce system load

Example: Temporarily removing a process from memory

3. Scheduling Diagram



S. No.	Long-Term Scheduler	Short-Term Scheduler	Medium-Term Scheduler
1	It is a Job Scheduler	It is a CPU Scheduler	It is a process swapping scheduler
2	It takes process from the job pool	It takes process from the ready state	It takes process from running or wait/dead state
3	Its speed is lesser than short-term scheduler	It is fastest among the two other schedulers	Its speed is in between long-term and short-term
4	It controls the degree of multiprogramming	It has less control over the degree of multiprogramming	It reduces the degree of multiprogramming

AfterAcademy

4. Scheduling Criteria

Scheduling criteria are used to evaluate and select the best scheduling algorithm.

1. CPU Utilization

- Keeps CPU as busy as possible
- Should be **maximized**

2. Throughput

- Number of processes completed per unit time
- Should be **maximized**

3. Turnaround Time

- Time taken from submission to completion
- Should be **minimized**

4. Waiting Time

- Time a process spends in ready queue
- Should be **minimized**

5. Response Time

- Time taken to respond to a request
- Important for interactive systems
- Should be **minimized**

5. Working of Process Scheduling

1. Processes are first stored in **job queue**
2. Long-term scheduler selects processes into memory
3. Processes enter **ready queue**
4. Short-term scheduler selects process for CPU
5. If process waits for I/O → moved to waiting queue
6. Medium-term scheduler may suspend/resume processes

6. Conclusion

Process scheduling is essential for efficient CPU utilization and system performance.
Schedulers and scheduling criteria together ensure that processes are executed fairly and efficiently.

Q4. Explain CPU Scheduling Algorithms: FCFS, SJF, Round Robin, Priority

1. Definition of CPU Scheduling Algorithms

CPU scheduling algorithms are methods used by the operating system to decide the order in which processes are selected from the ready queue and allocated the CPU for execution.

These algorithms aim to maximize CPU utilization, minimize waiting time, and improve system performance.

2. Types of Scheduling

Non-Preemptive Scheduling

- Once a process starts execution, it continues until completion or it blocks
- CPU is not taken away forcibly

Preemptive Scheduling

- The operating system can interrupt a running process
- CPU is reassigned to another process

3. CPU Scheduling Algorithms

1. FCFS (First Come First Serve)

FCFS is a scheduling algorithm in which processes are executed in the order of their arrival in the ready queue. The process that arrives first gets the CPU first, and once allocated, it runs until completion.

Advantages:

- Simple and easy to implement

Disadvantages:

- High waiting time
- Convoy effect

2. SJF (Shortest Job First)

SJF is a scheduling algorithm in which the process with the smallest CPU burst time is selected for execution first. It can be preemptive or non-preemptive.

Advantages:

- Minimum average waiting time

Disadvantages:

- Starvation of long processes
- Difficult to predict burst time

3. Round Robin (RR)

Round Robin is a preemptive scheduling algorithm in which each process is assigned a fixed time quantum. Processes are executed in cyclic order.

Advantages:

- Fair scheduling
- Suitable for time-sharing systems

Disadvantages:

- High overhead if time quantum is small
- Large quantum behaves like FCFS

4. Priority Scheduling

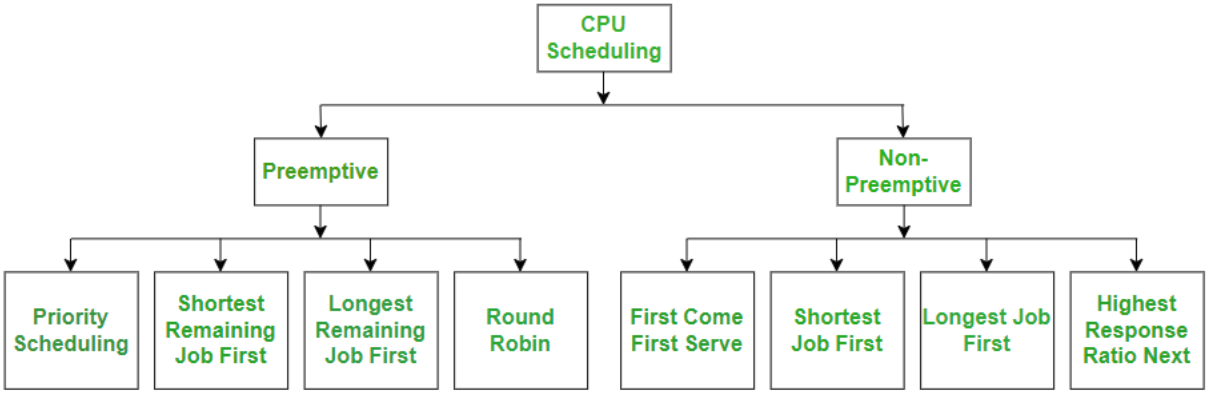
Priority scheduling assigns a priority to each process, and the CPU is allocated to the process with the highest priority. It can be preemptive or non-preemptive.

Advantages:

- Important processes are executed first

Disadvantages:

- Starvation of low priority processes
- Can be solved using aging



4. Comparison Table

Algorithm	Type	Key Idea	Problem
FCFS	Non-preemptive	Arrival order	Convoy effect
SJF	Both	Shortest job	Starvation
Round Robin	Preemptive	Time quantum	Overhead
Priority	Both	Priority-based	Starvation

5. Conclusion

CPU scheduling algorithms determine the execution order of processes in a system. Each algorithm has its own advantages and limitations, and the selection depends on system requirements such as efficiency, fairness, and response time.

Q5. Explain Context Switching

---

1. Definition

Context switching is the process in which the operating system stops executing one process, saves its current state, and loads the saved state of another process so that CPU execution can be switched between processes.

This allows multiple processes to share the CPU efficiently

---

2. Need for Context Switching

- Required in **multitasking systems**
- Allows multiple processes to run seemingly simultaneously

- Prevents one process from occupying CPU continuously
- Improves CPU utilization

---

3. What is Context?

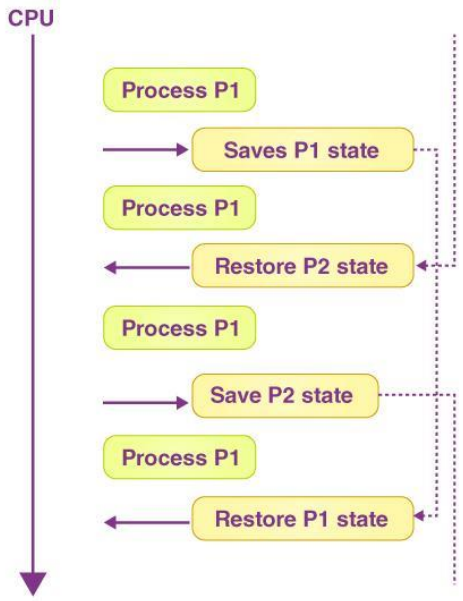
The **context** of a process includes:

- Program Counter
- CPU registers
- Process state
- Stack pointer

This information is stored in the **PCB (Process Control Block)**

---

4. Context Switching Diagram



5. Working of Context Switching

1. CPU executes process P1
2. Interrupt or time slice expiration occurs
3. OS saves P1 state into its PCB
4. Scheduler selects next process (P2)
5. OS loads P2 state from its PCB
6. CPU starts executing P2

This process repeats continuously.

---

6. When Context Switching Occurs

- Time quantum expires
- Interrupt occurs
- Higher priority process arrives
- Process requests I/O

---

7. Context Switching Overhead

- Time spent switching is called **overhead**
- During this time, CPU does no useful work
- Frequent switching reduces system performance

---

8. Conclusion

Context switching is a key mechanism in operating systems that enables multitasking by saving and restoring process states, ensuring efficient CPU sharing.

Q6. Define Thread and Explain Thread Models

1. Definition of Thread

A **thread** is the smallest unit of CPU execution within a process.
Multiple threads in a process share code, data, and resources, but each thread has its own program counter, registers, and stack.

Threads are also known as **lightweight processes**.

---

2. Thread Models

1. Many-to-One Model

- Many user threads are mapped to one kernel thread
- Managed at user level

Advantages:

- Simple and fast

Disadvantages:

- No parallelism
- If one thread blocks, all threads block

Example: User-level thread libraries

2. One-to-One Model

- Each user thread maps to one kernel thread

Advantages:

- Supports true parallel execution

Disadvantages:

- Higher overhead due to kernel involvement

Example: Modern operating systems

3. Many-to-Many Model

- Many user threads are mapped to many kernel threads

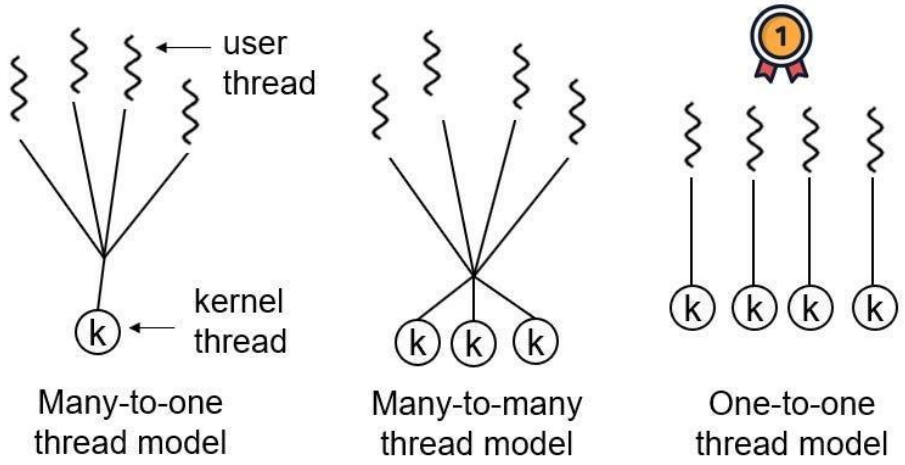
Advantages:

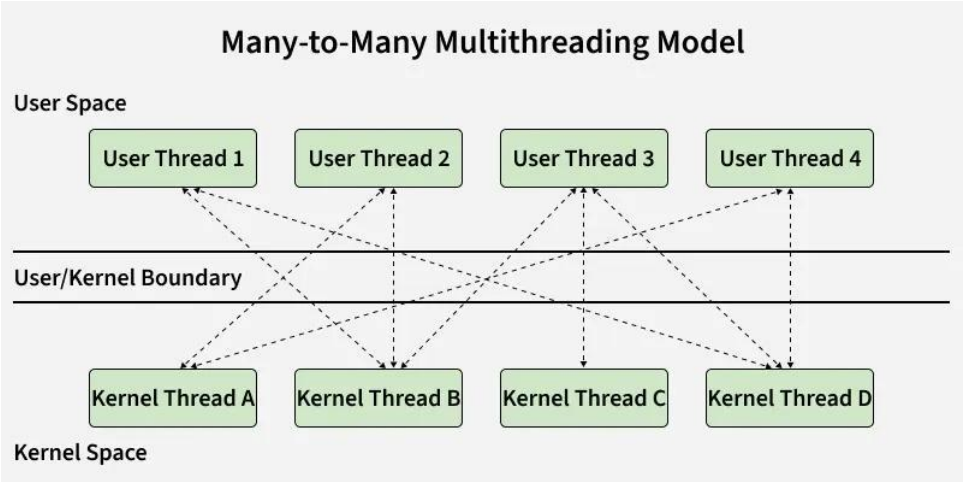
- Efficient and scalable
- Supports parallelism

Disadvantages:

- Complex to implement

3. Thread Model Diagram





4. Conclusion

Threads improve system performance by enabling concurrent execution within a process, and thread models define how threads are managed and executed efficiently.

Q7. Differentiate Between Process and Thread

Definition

A **process** is an independent program in execution that has its own memory space, resources, and control structures managed by the operating system.

A **thread** is the smallest unit of execution within a process. Multiple threads exist inside a process and share its memory and resources, but each thread has its own program counter, registers, and stack.

Comparison Table

Process	**Thread**
A process is an independent execution unit	A thread is a part of a process
Each process has its own separate memory space	Threads share the same memory space
Process creation and termination are time-consuming	Thread creation and termination are faster
Context switching between processes is slower	Context switching between threads is faster
Communication between processes is complex (IPC)	Communication between threads is simple
Processes are isolated from each other	Threads are not isolated (share resources)

Failure of one process does not affect others	Failure of one thread can affect entire process
Processes require more system resources	Threads require fewer resources
Process is a heavyweight entity	Thread is a lightweight entity

---

Conclusion

Processes provide isolation and reliability, while threads provide efficiency and faster execution. Threads are preferred in applications requiring concurrency, whereas processes are used when strong isolation is required.

Q8. Explain Interprocess Communication (IPC)

---

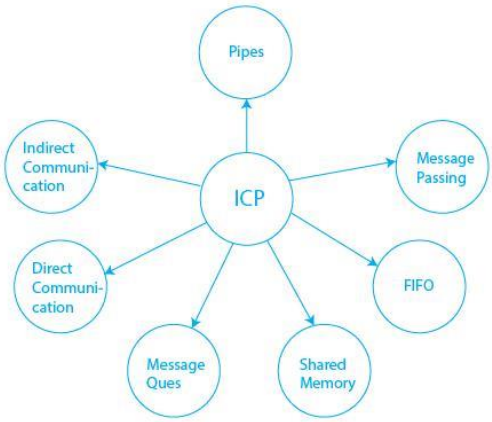
1. Definition

Interprocess Communication (IPC) is a mechanism provided by the operating system that allows processes to communicate with each other and synchronize their actions while executing.

Since processes run in separate memory spaces and cannot directly access each other's data, IPC provides controlled methods for data exchange and coordination, ensuring efficient and safe execution.

2. Objectives of IPC

- Information sharing between processes
- Process synchronization
- Resource sharing
- Improving system performance



Classical Problems in IPC

1. Dining Philosophers Problem

- Demonstrates deadlock and starvation
- Multiple processes compete for shared resources

2. Producer–Consumer Problem

- Deals with buffer sharing
- Producers add data, consumers remove data
- Requires synchronization

3. Readers–Writers Problem

- Multiple readers can read simultaneously
- Writers need exclusive access

4. Sleeping Barber Problem

- Models process coordination
- Handles waiting and resource allocation

6. Example

In a bank ATM system:

- One process reads input
- Another checks balance
- Another dispenses cash

Processes coordinate using IPC

---

7. Conclusion

IPC is essential for process communication and synchronization, enabling efficient multitasking and system performance.

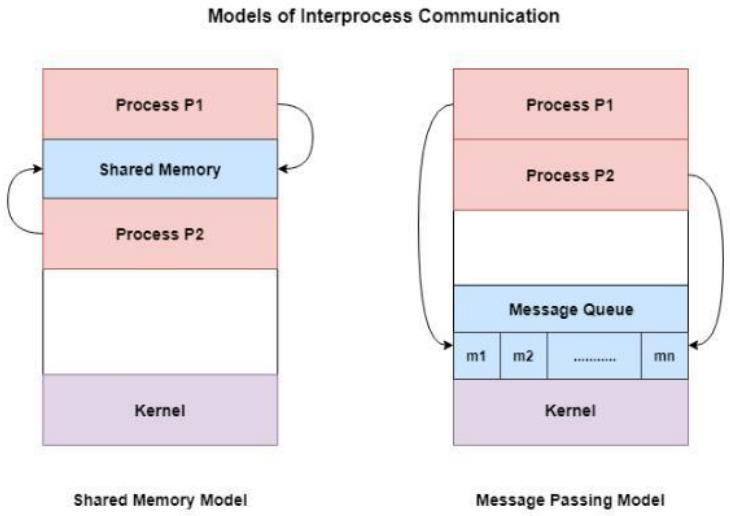
Q9. Compare Shared Memory and Message Passing

Shared Memory is an IPC method where multiple processes access a common memory region to exchange data directly.

Message Passing is an IPC method where processes communicate by sending and receiving messages through the operating system.

2. Comparison Table

Shared Memory	Message Passing
Processes share a common memory space	Processes exchange data via messages
Direct access to shared memory	Communication through OS (system calls)
Faster communication	Slower due to kernel involvement
Suitable for large data transfer	Suitable for small data transfer
Requires synchronization (semaphores)	No need for shared memory synchronization
Risk of data inconsistency	Safer communication
Less OS involvement after setup	High OS involvement
More complex to implement	Easier to implement
Working: • OS creates shared memory • Process A writes data • Process B reads data	**Working:** • Process A sends message • OS transfers message • Process B receives message



3. Conclusion

Shared memory is faster and efficient for large data transfer, while message passing is simpler and safer for communication between processes. The choice depends on system requirements.